SW Engineering CSC648-848 Fall 2025

SFSU Tutoring Website

Team 01

Milestone 1

Name	Role(s)
Amer Music - amusic@mail.sfsu.edu	Project Lead & Frontend Developer
Arric Sekhon	Frontend Lead
Derek Ye	Backend Lead
Rishika Bharodiya	GitHub Master
Stanley Wang	Backend Support
Alonzo Manosca	GitHub Support
Daniel Castillo	Frontend Support

Document History

	Submitted Date	Revised Date
Milestone 1	Oct 15, 2025	Oct 27, 2025

Table of Contents

Executive Summary 4
Personas 5
Persona 1 — Sophia Nguyen (Student Seeker) 5
Persona 2 — Marco Alvarez (Tutor) 6
Persona 3 — Jordan Lee (Administrator) 7
High-Level Use Cases 8
Course-Centric Tutor Discovery 8
Publish a Tutor Listing 8
Moderate Listings and Keep the Catalog Clean 8
One-Round In-Site Message with Clear Confirmation 8
SFSU-Specific: Semester-Aware Availability & Campus Location Match 8
List of Main Data Items and Entities — Data Glossary / Description 9
Key Users 9
Unregistered Student 9
Registered Student 9
Registered Tutor 9
Admin 9
Data Items 9
User Profile 9
Tutor Profile 9
Message 9
Admin Review Log 9
High-Level Functional Requirements 11
FR-1 SFSU Registration & Sign-in (@sfsu.edu) 11
FR-2 Tutor Role Enablement 11
FR-3 Create/Update Listing 11
FR-4 Admin Approval & Moderation 11
FR-5 Browse & Search (course-centric, data-driven filters) 11
FR-6 Listing Detail 11
FR-7 One-Round In-Site Messaging 11
FR-8 Role-Based Dashboards 11
FR-9 SFSU-Specific Discovery 11
FR-10 Analytics & Banner 11
Non-Functional Requirements (NFR) 12

Competitive Analysis 13
High-Level System Architecture and Technologies Used 14
Software Components and Versions 14
Deployment Cloud Service 14
Front-End Frameworks 14
Supported Browsers 14
External Open-Source APIs 14
Use of Gen AI Tools 14
Team and Roles 15
Team Lead Checklist 16

1.) Executive Summary

GatorMatch Tutors is an SFSU-only web application that makes it fast, safe, and credible for students to find peer tutors for the exact courses they're taking. Instead of bouncing across generic marketplaces and informal channels, any @sfsu.edu student can search by course code (e.g., CSC 340), see meeting options at a glance (Zoom or on-campus), review concise tutor profiles, and send a single in-site message to start contact—no email chase or chat threads. Every tutor profile clearly surfaces trust signals that matter to students and faculty: a 5-star rating generated by verified students who have used that tutor's services, the total hours tutored to date, and an eligibility badge confirming the tutor has completed the course with a minimum grade of 90% and/or holds a professor's recommendation. Tutors can create and manage listings with clear status (Draft, Pending, Approved), and every listing is reviewed by an administrator who can approve, reject with a reason, or later unpublish to keep the catalog clean and trustworthy.

What makes GatorMatch distinctive is its SFSU fit. Discovery is course-centric and campus-aware: listings are tagged with official course codes and campus locations such as the J. Paul Leonard Library, and availability is expressed in semester-friendly terms students actually use ("post-lab hours," "midterm week evenings"). Filters are data-driven so options with zero matches never waste a student's time. The result is a moderated, auditable directory that shortens time-to-help for students, channels relevant inquiries to tutors, and provides the university a safe, focused service without payments or external messaging.

2.) Personas

Persona 1 — Sophia Nguyen

	**About:** **Jordan reviews new tutor listings, checks course accuracy and appropriateness, and keeps the directory clean. Time is scarce; decisions must be quick and auditable.**

Role: Student Seeker (Primary) | Age: 20 | Location: Daly City ↔ SFSU (commuter) | Status: B.S. CS, 3rd year

"I just need the right tutor for CSC 340 and a quick way to contact them—no endless back-and-forth."

💡 **Motivations:**

- Find a tutor by course prefix + number in minutes
- See availability and meeting mode up front
- Keep everything inside the site (no email chase)

🎯 **Goals:**

- Results that match *CSC 340* with live counts on filters
- Scan tutor cards with course tags and Library/Zoom badges
- Send a single in-site message and get a clear confirmation

⚡ **Frustrations:**

- Irrelevant results when filtering by course
- Filters that show options with 0 matches
- Long forms or multi-step chat/DM outside the site

🖥️ **Demographics / Context:**

- Devices: 13" laptop; iPhone on the go
- Study window: late evenings; variable Wi-Fi
- Accessibility: prefers high contrast, large tap targets

🧭 **Primary Scenario (What, not How)**

Before midterms, Sophia searches for "CSC 340," filters results to Zoom, skims a few relevant tutor profiles, opens one that looks best, and sends a quick message with her request — all within the site.

Persona 2 — Marco Alvarez

	**About:** **Marco mentor's undergraduates in CSC 340 and CSC 648. He values a simple, efficient way to post listings with clear approval feedback. He wants inquiries that are relevant to his courses—no spam or long email threads.**

Role: Tutor (Provider) | Age: 24 | Location: San Francisco | Status: M.S. CS, 1st year

"Make it easy to publish a legit listing and let students contact me once with the right context."

💡 **Motivations:**

- Quick Tutor enablement after sign-up
- Clean form for headline, bio, course prefixes, availability, Zoom/campus tags
- Visibility in course-centric searches

🎯 **Goals:**

- Listing Submitted → Pending → Approved with reason if rejected
- Appear when students search by course code
- Receive a single, contextual in-site message

⚡ **Frustrations:**

- Ambiguous forms and unclear approval outcomes
- Off-topic inquiries ("Do you teach Math?" for a CSC listing)
- Being forced into email chains or outside chats

🖥️ **Demographics / Context:**

- Devices: Laptop to edit, phone to check messages
- Time: 20-minute gaps between classes
- Privacy: prefers in-site contact over personal email

🧭 **Primary Scenario (What, not How):**

After class, Marco enables Tutor role, adds *CSC 340/648*, Library & Zoom tags, sets availability, submits; later sees Approved and starts receiving targeted, one-round messages.

Persona 3 — Jordan Lee

	**About:** **Jordan reviews new tutor listings, checks course accuracy and appropriateness, and keeps the directory clean. Time is scarce; decisions must be quick and auditable.**

Role: Administrator (Moderator) | Age: 28–35 | Location: SFSU | Status: Staff / Appointed Moderator

"I need to triage listings fast with enough context to approve or reject—and maintain a clean catalog."

💡 **Motivations:**

- Approval Queue with compact snapshots (headline, bio, course tags, meeting mode)
- Approve / Reject with reason, and Unpublish/Delete later
- Basic moderation on users/content when needed

🎯 **Goals:**

- Process new listings within 24–48 hours
- Maintain audit trail (who/when/why)
- Keep catalog SFSU-appropriate and course-accurate

⚡**Frustrations:**

- Sparse listings lacking course/meeting tags
- No reason field or history of actions
- Needing multiple screens to decide

🖥️ **Demographics / Context:**

- Devices: Laptop; short review windows between duties
- Interaction style: bulk triage, keyboard driven
- Signals: clear status badges; queues/filters

Primary Scenario (What, not How) :

Jordan opens Approval Queue, scans listing snapshots, Approves good entries, Rejects with reason where needed, and later Unpublishes any reported listing.

3.) High-Level Uses Cases

Course-Centric Tutor Discovery
Actors: Sophia (Student Seeker)
Before midterms, Sophia wants help for CSC 340. She enters the course code, limits results to meeting modes that work for her (Zoom or on-campus), and quickly scans only relevant tutor cards that clearly show course tags, current availability, and a high average star-rating. She opens a profile she likes, confirms it matches her schedule and mode, and proceeds to contact that single tutor without needing external email or multi-step chats. If no good match appears, she adjusts filters that only show options with real matches, so she doesn't waste time.

Publish a Tutor Listing
Actors: Marco (Tutor)
Between classes, Marco decides to offer tutoring for CSC 340 and CSC 648. He enables the Tutor role by creating a **tutor profile** with his SFSU email and completes a concise listing that includes a headline, a short bio, where he can meet (Zoom or on-campus), and 1-hour availability slots. He adds only courses he has completed with a grade of ≥90% and uploads up to three verification documents (e.g., transcript screenshots) that match those courses. After certifying the policies, he submits the listing and watches the status progress from Submitted to Under Review to Approved (or Changes Requested). Once approved, his listing becomes discoverable in course-centric searches so inquiries he receives are on-topic and include useful course context.

Moderate Listings and Keep the Catalog Clean
Actors: Jordan (Administrator)
During a short review window, Jordan opens the approval queue and sorts any new tutor submissions. For each, Jordan checks that course tags, bio, and meeting modes are appropriate for SFSU. Good entries are approved; unclear or off-scope entries are rejected with a brief reason, so tutors know how to improve. Later, if a listing is reported, Jordan can unpublish or remove it and maintain an auditable record of actions for catalog transparency and ensure that it is SFSU-appropriate. Unclear or off-scope entries are rejected by selecting predefined reason(s) from a multi-select list and, when needed, adding a short note so tutors know exactly how to improve.

One-Round In-Site Message with Clear Confirmation
Actors: Sophia (Student Seeker), Marco (Tutor)
After opening Marco's profile, Sophia sends one concise, in-site **message** that includes course, topic focus, and preferred meeting mode. She immediately sees that the **message** was sent; there's no expectation of a threaded chat. Marco reviews the **message** in his inbox during a break and has the necessary context to accept or propose a time through his chosen channel or meeting mode outside the system. Both can later reference the single **message** in their role-based dashboards (Sent/Inbox) without exposing personal emails.

SFSU-Specific Discovery: Semester-Aware Availability & Campus Location Match *(differentiator)*
Actors: Sophia (Student Seeker), Marco (Tutor)
Sophia needs someone available around SFSU's semester rhythm and prefers to meet at the J. Paul Leonard Library. She filters by course code and selects campus location tags to see tutors who explicitly meet at SFSU spaces. Availability is expressed in semester-aware terms students recognize (e.g., "post-lab hours," "midterm week evenings"), so she can align help with real campus schedules.

Semester-aware phrases are saved as lightweight tags per course/section and automatically resolved into real time windows using the term and course calendars (labs, lectures, midterm weeks). At search time, these windows are intersected with the tutor's existing hourly slots and location tags, keeping results accurate without changing the current calendar UI. This SFSU-specific combination of course-centric search, campus location tags, and semester-aware availability guides her to a fit she couldn't easily find on generic tutoring sites.

4.) List of Main Data items and Entities – Data Glossary/Description

- Key Users
  - Unregistered Student

Definition	Any user who visits the platform without logging in
Permissions	Can browse and search tutor listings, but cannot send messages, create or edit listings, or access dashboards
Usage	Enables casual browsing before sign-up, encouraging new users to register once they find relevant tutors

  - Registered Student

Definition	A verified SFSU student logged into the system to find tutoring help
Permissions	Can browse/search tutor listings and send a one-time message to tutors
Usage	Primary consumer of tutoring services; uses the platform to find and contact peer tutors

  - Registered Tutor

Definition	A verified SFSU student providing tutoring services through the platform
Permissions	Can create and update tutor listings; submit listings for admin approval
Usage	Primary provider of tutoring services; maintains visibility and availability through listings

  - Admin

Definition	A user responsible for managing and moderating the platform
Permissions	Can view all tutor listings, approve or reject posts (with reasons), unpublish or delete content, and moderate users
Usage	Ensures platform quality, verifies listings, and maintains a safe environment for all users

- Data Items
    - User Profile

Definition	Stores verified SFSU user information and determines the user's role (Student, Tutor, or Admin)
Sub Items	• Full Name • SFSU Email (@sfsu.edu) • Role (Student, Tutor, Admin) • Profile Picture (optional) • Registration Date • Account Status (Active / Blocked)
Input	Created during registration and updated by the user
Usage	Used for authentication, access control, and displaying role-based dashboards

    - Tutor Profile

Definition	A profile-like listing created by a tutor that advertises their services
Sub Items	• Tutor Name • Subject Expertise (e.g., CSC 648, MATH 226) • Availability (day and time) • Campus Location Tag (Library, Zoom, etc.) • Additional Notes / Description • Status (Pending, Approved, Rejected) • Admin Review Reason (optional)
Input	Created and submitted by Tutor; requires Admin approval before being public
Usage	Displayed in searchable listings for Students; visibility controlled by approval status

    - Message

Defintion	A one-round, in-site message sent from a Student to a Tutor regarding a specific listing
Sub Items	• Message ID • Sender (Student) • Receiver (Tutor) • Linked Listing ID • Message Content • Timestamp • Status (Sent, Read)
Input	Created automatically when a Student sends a one-round message to a Tutor
Usage	Appears in the Student's "Sent" box and Tutor's "Inbox"; no replies or external email allowed

- Admin Review Log

Definition	A record of all Admin actions taken on Tutor Posts or Messages for accountability and moderation
Sub Items	• Admin ID • Action Type (Approve, Reject, Unpublish, Delete) • Target Post / Message ID • Timestamp • Reason / Notes
Input	Automatically generated when Admin performs an action
Usage	Enables transparent moderation tracking and supports audit or rollback if needed

5.) High-Level Functional Requirements

- **FR-1 SFSU Registration & Sign-in:** Users must register/sign in with @sfsu.edu (suffix check only).
- **FR-2 Tutor Role Enablement:** Students can enable a Tutor role to create listings.
- **FR-3 Create/Update Listing:** Tutor can add/edit headline, bio, subjects, course prefixes, availability, location tags, optional display-only rate.
- **FR-4 Admin Approval & Moderation:** Admin can approve/reject (with reason), unpublish, delete listings; moderate users/content.
- **FR-5 Browse & Search:** Search and filter by Term, Session, Subject, Course Number, Location; options are data-driven and disabled/hidden when there are 0 matches.
- **FR-6 Listing Detail:** Detail page shows bio, subjects, course prefixes, availability, location tags, and Contact Tutor.
- **FR-7 One-Round In-Site Messaging**: Student can send one message to a tutor; no replies; no email clients.
- **FR-8 Role-Based Dashboards:** Student (Sent, profile); Tutor (Listings, Inbox, profile); Admin (Approval Queue, Moderation).
- **FR-9 SFSU-Specific Discovery:** Course-centric filters, campus location tags, and semester-aware availability text.
- **FR-10 Analytics & Banner:** Google Analytics enabled; exact required banner text on every page.

6.) Non-Functional Requirements (NFR)

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0
2. Application shall be optimized for standard desktop/laptop browsers e.g must render correctly the two latest versions of two major browsers.
3. All or selected application functions shall be rendered well on mobile devices (no native app to be developed)
4. Posting of tutor information and messaging to tutors shall be limited only to SFSU students
5. Critical data shall be stored in the database on the team;s deployment server.
6. No more than 50 concurrent users shall be accessing the application at any time.
7. Privacy of users shall be protected.
8. The language used shall be English (no localization needed)
9. Application shall be very easy to use and intuitive.
10. Application shall follow established architecture patterns.
11. Application code and its repository shall be easy to inspect and maintain.
12. Google analytics shall be used.
13. No e-mail clients shall be allowed. Interested users (clients) can only message service providers via in-site messaging. One round of messaging (from client to service provider) is enough for this application. No chat functions shall be developed or integrated.
14. Pay functionality (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. Site security: basic best practices shall be applied (as covered in the class) for main data items.
16. Media formats shall be standard as used in the market today.
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools.
18. The application UI (WWW and mobile) shall prominently display the following exact test on all pages *"SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only"* at the top of the WWW page Nav bar. (Important so as to not confuse this with a real application).

7.) Competitive Analysis

Feature	Khan Academy	Wyzant	Preply	Skooli	Our site
Tutor profiles	-	+	+	+	+
Course searching	+	+	+	+	++
Schedule tutoring session	-	+	+	+	++
Dark mode	-	-	-	-	+
Tutor messaging	-	+	+	+	++

For our tutoring website, we plan on improving on the course search, tutor scheduling, and the tutor messaging systems that more popular tutoring websites have. We want to improve on the search feature by adding a student rating next to a certain tutor or course. Sort of like a built-in RateMyProfessor, but for tutors! To improve on the scheduling system that the popular websites have, we would like to add a feature that lets students put in their schedule and then our website picks the best times for both the student and the tutor. To keep communication simple and controlled, students may send one message to a tutor outside of a tutoring session; tutors cannot reply in the system, and students cannot send follow-ups. Any discussion occurs only during scheduled sessions. Email is not used for messaging: the platform does not send or receive emails, forward messages to email, or expose email addresses. Messages are viewable to the recipient tutor in-app only, and if a response is required, tutors will address it during the next session (or the student can book a session).
As a little bonus we want to add a native dark mode to our website that can be toggled on and off!

8.) High-Level System Architecture and Technologies Used

1. List all main software components and versions (DB, WWW server)

- **Operating System:** Ubuntu 22.04 LTS
- **Web Server:** NGINX 1.18.0 + Gunicorn 23.0.0
- **Database:** MySQL 8.0.37
- **Programming Language:** Python 3.10.18
- **Framework:** Flask 3.2.1 (used to build the web application) + Jinja2 3.1.6 (template engine)
- **Version Control:** Git + GitHub
- **Dependency Management:** requirements.txt file + pip 25.2
- **Front End:** HTML5 + CSS3 (Custom styles), Vanilla JS (for interactions)

2. List deployment cloud service you plan to use
We are deploying our web application on **Amazon Web Services (AWS)** using an **EC2 t3.micro** instance.

3. List front-end frameworks you will use
The site uses Flask-rendered HTML templates (Jinja templates) with custom CSS, no separate front-end framework.
4. List browsers you plan to support (choose 2 market leading browsers, last two versions from each)
We plan to support the following browsers:

- **Google Chrome** – 142 & 141
- **Safari** – 26.0.1 & 26.0

This will make sure our tutoring website works well for most users on both Windows and macOS.

5. List any major additional external open-source APIs you plan to use
We plan to use the following APIs in our project:

- **Google Analytics** – to track website visitors and understand user activity.
- **Zoom API** – to create and manage tutoring session links for students and tutors.
- **Google Maps API** – to show tutoring locations or directions (for future updates).

9.) Use of Gen AI Tools (ChatGPT, Copilot, etc.)

Our team didn't really rely on any GenAI tools for content generation for this milestone. Most of the work done for this milestone involved team discussion and individual writing. However we could see how these tools could be helpful for other teams when it comes to brainstorming personas, use case narratives or generating some ways to format certain sections. For future milestones we might explore GenAI tools a little more once we start developing and documenting more code.

10.) Team and Roles

Name	Eamil	Role
Amer Music	amusic@mail.sfsu.edu	Team Lead
Arric Sekhon	asekhon@sfsu.edu	Frontend Lead
Derek Ye	dye3@sfsu.edu	Backend Lead
Rishika Bharodiya	rbharodiya@sfsu.edu	Github Master
Stanley Wang	hwang26@sfsu.edu	Backend Support
Alonzo Manosca	amanosca1@sfsu.edu	Github Support
Daniel Castillo	dcastillo8@sfsu.edu	Frontend Support

11.) Team Lead Checklist

Checklist Item	**Status**	**Comments**
So far, all team members are fully engaged and attending team sessions when required	On Track (on-going)	Consistently engaged via discord & attending classes.
Team found a time slot to meet outside of the class	Done	Fridays at 5:30 pm
Team ready and able to use the chosen back and front-end frameworks and those who need to learn are working on learning and practicing	On Track (on-going)	
Team reviewed class slides on requirements and use cases before drafting Milestone 1	Done	
Team reviewed non-functional requirements from “How to start...” document and developed Milestone 1 consistently	Done	
Team lead checked Milestone 1 document for quality, completeness, formatting and compliance with instructions before the submission	On Track (on-going)	
Team lead ensured that all team members read the final M1 and agree/understand it before submission	Done	

Team shared and discussed experience with GenAI tools among themselves	On Track	
Github organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.)	On Track (on-going)	